SmartClass

Hosting Guide

Put SmartClass on a real server with a real domain and HTTPS

Version 1.1 | 27 October 2026

Three ways: A. cPanel shared hosting B. Ubuntu server with Apache C. Ubuntu server with nginx

Contents

1 Choose your way **2**

2 Prepare once (all ways) **2**

3 Way A — cPanel shared hosting **2**

4 Way B — Ubuntu server with Apache **3**

4.1 1. Secure the server . . . 3
4.2 2. Install the software . . . 3
4.3 3. Database . . . 4
4.4 4. Upload and unpack the release . . . 4
4.5 5. Private settings and permissions . . . 4
4.6 6. PHP and Apache . . . 4
4.7 7. HTTPS (free certificate) . . . 4
4.8 8. Install the application . . . 4
4.9 9. Cron jobs . . . 5

5 Way C — Ubuntu server with nginx **5**

6 After installation (all ways) **5**

6.1 If a proxy or Cloudflare is in front . . . 5
6.2 Settings to do first (as administrator) . . . 5

7 Pre-flight check **6**

8 Web-exposure check **6**

9 Backups — set up and test **6**

10 Upgrade **6**

11 Several schools on one server **7**

12 Troubleshooting **7**

1 Choose your way

	Good for	You need
A. cPanel	one school, no server skills, cheapest	a hosting plan with PHP 8.0+, MySQL/MariaDB, cron jobs, SSL (AutoSSL), 1 GB+ disk
B. Ubuntu + Apache	full control, several schools	a VPS (2 GB RAM is enough for one school), a domain, SSH
C. Ubuntu + nginx	same as B, lighter server	the same

Every way ends with the same three things: the **pre-flight check** (section 7), the **web-exposure check** (section 8) and a **tested backup** (section 9). Do not let students in before all three are green.

2 Prepare once (all ways)

1. **Make the release package** on your own computer, inside the project folder: `sh bin/package.sh`. It creates `../smartclass-release-DATE.tar.gz` without `.git`, tests, QA files, secrets and uploads. (Do *not* upload the project folder by hand and do not `git clone` into the web folder.)
2. **Domain:** point the domain's *A record* to the server's IP (VPS) or use the one your host gives you (cPanel). Example used below: `school.example.com`.
3. **Write down, in a password manager:** the database name, the database user, a long random database password, and a `secret_key` (create with `php -r 'echo bin2hex(random_bytes(32));'` or any 64-character random text). Never put them in the code or send them by chat.

3 Way A — cPanel shared hosting

1. **PHP version:** cPanel → **Select PHP Version**: choose 8.1 or newer. Tick the extensions `pdo_mysql`, `mbstring`, `curl`, `gd`, `zip`, `fileinfo`, `openssl`. In **Options** set `upload_max_filesize` = 14M and `post_max_size` = 20M.
2. **Database: MySQL Databases** → create a database, a user with the long password, and **add the user to the database** with **ALL PRIVILEGES**. cPanel adds a prefix: the real names look like `cpuser_smartclass` and `cpuser_sc`.
3. **Upload: File Manager** → the folder `public_html` (or the folder of the domain) → **Upload** the `.tar.gz` → select it → **Extract**. The folder must now contain `setup.php`, `pages/`, `config/`.
4. **Private settings:** in `config/` copy `local.example.php` to `local.php` and edit it:

```
<?php
return [
```

```
  'db_host'    => 'localhost',
  'db_name'    => 'cpuser_smartclass',
  'db_user'    => 'cpuser_sc',
  'db_pass'    => 'THE_LONG_DATABASE_PASSWORD',
  'secret_key' => 'THE_64_CHARACTER_RANDOM_TEXT',
  'base_path'  => '',        // '' = the site is at the domain root
];
```

Set the file permission to 640.

5. **Folders:** create `uploads`, `sessions`, `backups`, `logs`, `instances` if they do not exist (permission 755; if uploads fail later, use 775).
6. **HTTPS: SSL/TLS Status** → run AutoSSL for the domain. Open `https://school.example.com/setup.php`.
7. **Install:** press **Install Database**, then create the first teacher (this account is the administrator). Then **delete `setup.php`** in File Manager.
8. **Cron jobs: Cron Jobs** → add the four lines of `deploy/smartclass.cron` (use the PHP path of your host, for example `/usr/local/bin/php /home/cpuser/public_html/cron/send_` the schedules are in the file.
9. Continue with section 6.

Careful

On cPanel the `.htaccess` files do the protecting. If your host runs nginx only (no `.htaccess`), use way C rules in the host's nginx settings or ask support to deny `config/`, `includes/`, `.git` and `uploads/`. The exposure check in section 8 tells you whether it works.

4 Way B — Ubuntu server with Apache

Commands run on the server as a user with `sudo`; steps were written for Ubuntu 22.04/24.04 (the configuration files were prepared and the deny rules tested; the package installation itself you run on your server).

4.1 1. Secure the server

```
sudo apt update && sudo apt -y upgrade
sudo ufw allow OpenSSH && sudo ufw allow 'Apache Full' && sudo ufw enable
```

Use SSH keys, not passwords (`PasswordAuthentication no` in `/etc/ssh/sshd_config`).

4.2 2. Install the software

```
sudo apt -y install apache2 mariadb-server php8.3-fpm php8.3-mysql php8.3-mbstring \
     php8.3-curl php8.3-gd php8.3-zip php8.3-xml certbot python3-certbot-apache
sudo mysql_secure_installation
```

(If your Ubuntu has another PHP version, use its number everywhere.)

4.3 3. Database

```
sudo mysql
CREATE DATABASE smartclass_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'smartclass'@'localhost' IDENTIFIED BY 'THE_LONG_DATABASE_PASSWORD';
GRANT ALL PRIVILEGES ON smartclass_db.* TO 'smartclass'@'localhost';
FLUSH PRIVILEGES;
```

The installer needs no other right, because the database already exists.

4.4 4. Upload and unpack the release

```
scp smartclass-release-DATE.tar.gz you@SERVER:/tmp/
sudo mkdir -p /var/www/smartclass && cd /var/www/smartclass
sudo tar -xzf /tmp/smartclass-release-DATE.tar.gz
sudo mkdir -p uploads sessions backups logs instances config/instances
```

4.5 5. Private settings and permissions

```
sudo cp config/local.example.php config/local.php
sudo nano config/local.php      # db_name, db_user, db_pass, secret_key, 'base_path' =>
     ''
sudo chown -R www-data:www-data uploads sessions backups logs instances config/
    instances
sudo chmod 640 config/local.php && sudo chgrp www-data config/local.php
```

4.6 6. PHP and Apache

```
sudo cp deploy/php-smartclass.ini /etc/php/8.3/fpm/conf.d/90-smartclass.ini
sudo systemctl restart php8.3-fpm
sudo cp deploy/apache-smartclass.conf /etc/apache2/sites-available/smartclass.conf
sudo nano /etc/apache2/sites-available/smartclass.conf    # ServerName
sudo a2enmod proxy_fcgi setenvif rewrite headers && sudo a2enconf php8.3-fpm
sudo a2dissite 000-default && sudo a2ensite smartclass
sudo apache2ctl configtest && sudo systemctl reload apache2
```

4.7 7. HTTPS (free certificate)

```
sudo certbot --apache -d school.example.com
```

Certbot installs the certificate, the redirect to HTTPS and automatic renewal. Test renewal: `sudo certbot renew --dry-run`.

4.8 8. Install the application

Open `https://school.example.com/setup.php` → **Install Database** → create the first teacher → then `sudo rm /var/www/smartclass/setup.php`.

4.9 9. Cron jobs

```
sudo crontab -u www-data -e      # paste deploy/smartclass.cron (check the path)
```

Continue with section 6.

5 Way C — Ubuntu server with nginx

Do steps 1, 3, 4, 5, 8 and 9 of way B, but install nginx instead of Apache:

```
sudo apt -y install nginx mariadb-server php8.3-fpm php8.3-mysql php8.3-mbstring \
     php8.3-curl php8.3-gd php8.3-zip php8.3-xml certbot python3-certbot-nginx
sudo ufw allow 'Nginx Full'
sudo cp deploy/php-smartclass.ini /etc/php/8.3/fpm/conf.d/90-smartclass.ini && sudo
    systemctl restart php8.3-fpm
sudo cp deploy/nginx-smartclass.conf /etc/nginx/sites-available/smartclass
sudo nano /etc/nginx/sites-available/smartclass            # server_name, php socket
sudo ln -s /etc/nginx/sites-available/smartclass /etc/nginx/sites-enabled/
sudo rm /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d school.example.com
```

nginx ignores `.htaccess`; the block in `deploy/nginx-smartclass.conf` contains the equivalent rules (deny `.git`, `config`, `includes`, `tests`, `backups`, private upload folders; never execute anything in `uploads/`; 20 MB bodies; unbuffered live AI answers). It was syntax-checked with `nginx -t` and its deny rules were tested with `curl`.

6 After installation (all ways)

6.1 If a proxy or Cloudflare is in front

When another server ends HTTPS in front of SmartClass (Cloudflare, a load balancer, a hosting panel proxy), add to `config/local.php`: `'trust_proxy' => true,`. Without it the site would build `http://` links and session cookies without the Secure flag. Do this *only* when you control that proxy.

6.2 Settings to do first (as administrator)

1. **Settings** → **Personal** → **Two-step sign-in**: switch it on for yourself.
2. **Communication** → **E-mail**: SMTP host, port, user, password of your mail account (stored encrypted) and press the test button. Without it nobody can reset a password.
3. **Rules & policies** → security rules: switch on **Force HTTPS** (possible only from a page opened over HTTPS).
4. School name, logo, languages, grading scale.

7 Pre-flight check

On the server, in the project folder:

```
php bin/preflight.php
```

It prints PASS, WARN or FAIL for: PHP version and extensions, `config/local.php` and the secret key, database login and connection, all migrations applied, an administrator exists, no demo account with a well-known password, Force HTTPS, e-mail, cron jobs ran, folders, `setup.php` removed, no `.git/tests` in the web folder. It changes nothing. **Every FAIL must be fixed; read every WARN.** Run it again after every upgrade. (Folders may show a WARN when you run it as your own user instead of the web user; check them in **Settings** → **System** → **System check**.)

8 Web-exposure check

From *another computer* (replace the domain). Every line must **not** print 200:

```
for p in .git/config config/local.php includes/auth.php tests/lib.php \
         logs/php-error.log backups/ sessions/ setup.php README.md; do
  printf "%-26s " $p; curl -s -o /dev/null -w "%{http_code}\n" https://school.example.
    com/$p
done
curl -sI http://school.example.com/ | head -3     # must redirect (301) to https
curl -sI https://school.example.com/pages/auth/login.php | grep -i "set-cookie\|x-frame
    \|nosniff"
```

Expected: 403 or 404 for every path (`setup.php` must be 404 after you deleted it). The login page must set a cookie with `Secure; HttpOnly; SameSite=Lax`. Also upload a 5 MB PDF as a test resource: if it fails, the PHP limits (`upload_max_filesize`, `post_max_size`) were not applied to the web PHP.

9 Backups — set up and test

1. The cron lines of `deploy/smartclass.cron` make a nightly database backup into `backups/all/` (14 days) and copy the newest one to *another machine* (edit the `--to=` target; SSH key login required). A backup only on the same server does not survive a lost server.
2. Copy `uploads/` as well (it is not in the database backup): `rsync -a /var/www/smartclass/uploads/ user@backup-host:/backups/smartclass-uploads/` nightly.
3. **Test it:** `php bin/restore_drill.php` — it restores the newest backup into a temporary database, compares every table and foreign key, drops the copy and prints `restore drill PASSED`.

10 Upgrade

1. On your computer: `git pull`, `sh bin/check.sh` (all tests must pass), `sh bin/package.sh`.

2. On the server: backup (`php bin/backup_all.php`); unpack the new package *over* the folder (it never contains `config/local.php`, `uploads/`, `backups/`, `config/instances/`); fix owners (`chown -R www-data:www-data uploads sessions backups logs`).
3. `php bin/migrate.php` (or `php bin/update_all.php` for hosted schools), then `php bin/preflight.php`.

11 Several schools on one server

`php bin/create_customer.php --name="Example School" --domain=school2.example.com --admin-email=owner@school2.example.com` creates its own database, user, storage folder and configuration, and prints the administrator's temporary password once. Point the domain to the same server and add it to the web server (Apache: another `ServerName` / `ServerAlias`; nginx: add the domain to `server_name`), then run certbot for it. Check one school with `php bin/preflight.php school2.example.com`.

12 Troubleshooting

Symptom	Check
Database error on every page	`config/local.php` exists and is right; `mysql -u smartclass -p smartclass_db` works; `php bin/preflight.php`.
Blank page / 500	last lines of `logs/php-error.log` (pages never print errors).
Links start with `http://` on an HTTPS site	a proxy ends HTTPS: set `'trust_proxy' => true` (section 6).
Links or images broken	`'base_path'` in `config/local.php`: `''` for a domain root, `'/folder'` for a sub-folder.
Uploads fail	`uploads/` owner/permissions; PHP limits; disk space.
No e-mail	e-mail not set up or the cron line for `send_emails.php` missing.
Live AI answers arrive all at once	nginx: `fastcgi_buffering off` (included in the block); a proxy in front must not buffer.
`/.git/config` answers 200	Apache: `AllowOverride All` missing (the `.htaccess` is ignored); nginx: the deny block is not active.